

3

Discrete Event Simulation (DES) Systems



Discrete Event Simulation (DES)

- Modeling of a system as it evolves over time
- The simulation must provide:
 - a representation of the **state** of the physical system (*dependent variable*)
 - the state is defined as a collection of **state variables**, namely program variables usually specified in some high-level programming language such as C or Java that represents the state of the physical system
 - some means of changing this representation to model the evolution of the physical system
 - changes in the state of the physical system are realized by the simulation program writing new values into these state variables
 - a representation of **time** (*independent variable*)
 - the time is represented through an abstraction called **simulation time**
- More precisely, state can change at only a countable number of points in time
 - These points in time are when events occur
- **Event**: Instantaneous occurrence that *may* change the state of the system

Other types of simulation

- *Continuous simulation*

- Typically, solve sets of differential equations numerically over time
- May involve stochastic elements
- Some specialized software available; some discrete-event simulation software will do continuous simulation as well

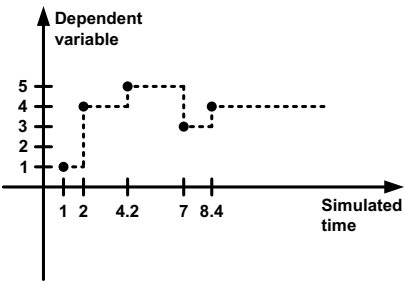
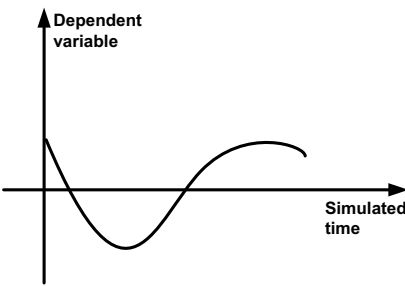
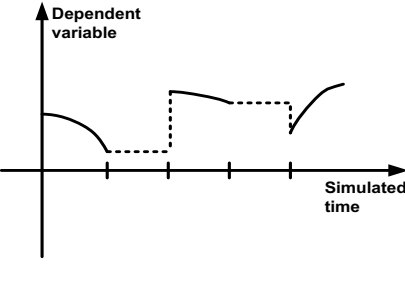
- *Combined discrete-continuous (hybrid) simulation*

- Continuous variables described by differential equations
- Discrete events can occur that affect the continuously-changing variables
- Some discrete-event simulation software will do combined discrete-continuous simulation as well

- *Monte Carlo simulation*

- No time element (usually)
- Wide variety of mathematical problems

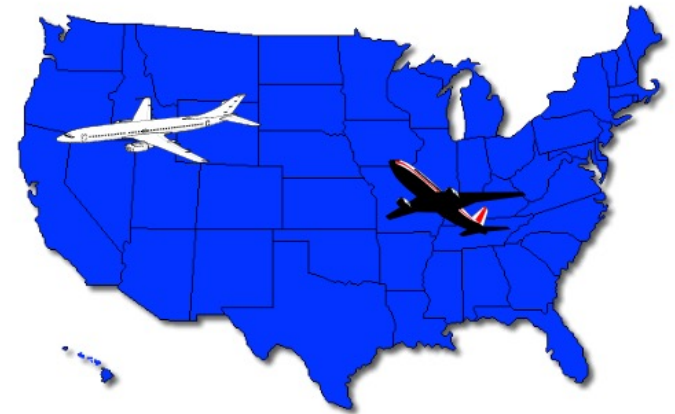
Types of simulation *wrt* state representation

Discrete Simulation	 A graph showing a dependent variable on the y-axis (ranging from 1 to 5) against simulated time on the x-axis. The function is a step function, represented by a dashed line with dots at the corners. The steps occur at time values 1, 2, 4.2, 7, and 8.4. The values of the function are 1, 4, 5, 3, and 4 respectively for these intervals.
Continuous Simulation	 A graph showing a dependent variable on the y-axis against simulated time on the x-axis. The function is a smooth, continuous curve, represented by a solid line, oscillating above and below the x-axis.
Combined Simulation	 A graph showing a dependent variable on the y-axis against simulated time on the x-axis. The function is a piecewise function, represented by a dashed line with solid curves at the transitions. The function starts with a solid curve, then transitions to a dashed step function, and finally transitions back to a solid curve.

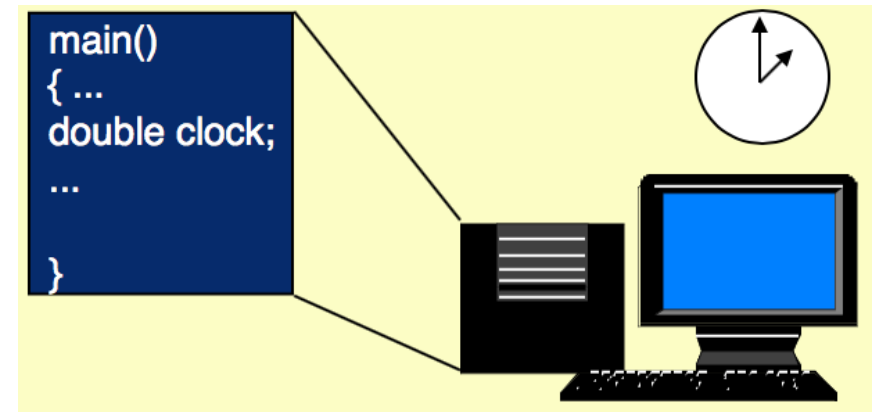
The time notion

There are several different notions of time that are important when discussing a simulation:

- **Physical time:** refers to time in the physical system
- **Simulation time:** is an abstraction used by the simulation to model physical time
- **Wallclock time**
 - refers to time during the execution of the simulation program
 - a simulation program can usually obtain the current value of wallclock time (accurate to some specifiable amount of error) by reading a hardware clock maintained by the operating system



physical system



Execution time: cpu time required for completing the simulation

The different time notions

To illustrate these different notions of time, consider a simulation of the transportation system in Atlanta during the 1996 summer Olympic games:

- *Physical time* for this simulation extends from July 19 to August 4, 1996 (17 days)
- *Simulation time* might be represented in the simulation program by a double precision floating point number **with each unit corresponding to a single day**. In this case, simulation time advances from 0.00 to 17.00 during each execution of the program
- If the simulation program ran for three hours on the afternoon of February 25, 1995, while planning for the Olympics, *wallclock time* might extend from 3:00 PM to 6:00 PM on that day

Simulation Time

- A new concept of time, which only exists in simulated worlds
- Defined as follows:
 - a totally ordered set of values where each value represents an instant of time in the physical system being modeled
 - for any two values of simulation time T_1 representing physical time P_1 , and T_2 representing P_2 :
 - if $T_1 < T_2$, then P_1 occurs before P_2 [9.0 represents 9 PM, 10.5 represents 10:30 PM]
 - and $(T_2 - T_1)$ is equal to $(P_2 - P_1) * K$ for some constant K [1.0 in simulation time represents 1 hour of physical time]
 - if $T_1 < T_2$, then T_1 is said to occur *before* T_2 , and if $T_1 > T_2$, then T_1 is said to occur *after* T_2
- The *linear relationship* between intervals of simulation time and intervals of physical time ensures durations of simulation time have a proper correspondence to durations in physical time
- For any given simulation, one common, **global simulation time scale** is used that is recognized by all components of the simulation, just as all countries in the world recognize GMT (Greenwich Mean Time)

Simulation time vs. Wallclock time

The progression of simulation time during the execution of the simulation may or may not have a direct relationship to the progression of wallclock time:

W2S denotes that T_s is a function (wallclock to simulation time) of the wallclock time.

- **Real-time simulation**

- simulation executions where advances in simulation time are paced by wallclock (for, e.g., virtual environments or training exercises)

- **Scaled real-time simulation**

- simulation time advances *faster* or *slower* than wallclock time by some constant factor (similar to fast-forward and slow motion, respectively)

$$T_s = W2S(T_w) = T_{start} + Scale * (T_w - T_{wStart})$$

where where T_s is the simulation time, T_w is a value of wallclock time, T_{start} is the simulation time at which the simulation begins, T_{wStart} is the wallclock time at the beginning of the simulation, and $Scale$ is the scale factor

As-fast-as-possible simulation

- In *simulations* that do not include humans or physical devices as components within the simulation (e.g., simulation for system analysis), the progression of simulated time is usually not paced by wallclock time
- A unit of advance in simulated time could require a few seconds in wallclock time during one part of the simulation, minutes in another part, or even hours in a third
- These simulations are sometimes referred to as **as-fast-as-possible simulations** because one wishes to complete the execution of the simulation as quickly as possible
- The relationship between the rate of advance in simulation time with advances in wallclock time bears no significance, except to the extent that it provides a metric of how long the user has to wait to get the results of the simulation (i.e., the wallclock time that takes the simulation execution)

Level of detail of a simulation model

- Metrics that define the level of detail
 - Important to identify a tradeoff between the level of detail (accuracy) and the cost of simulation (which can be complex in terms of space and time and thus cost-demanding)
 - depends on the objectives of the simulation
- **R ratio**
 - between the **execution time** – *or wallclock time required to complete the execution of the simulation* – and the **simulation time**
$$R = \frac{\text{execution time}}{\text{simulation time}}$$
- **Resolution time T**
 - the **lowest simulation time interval** between two consecutive events

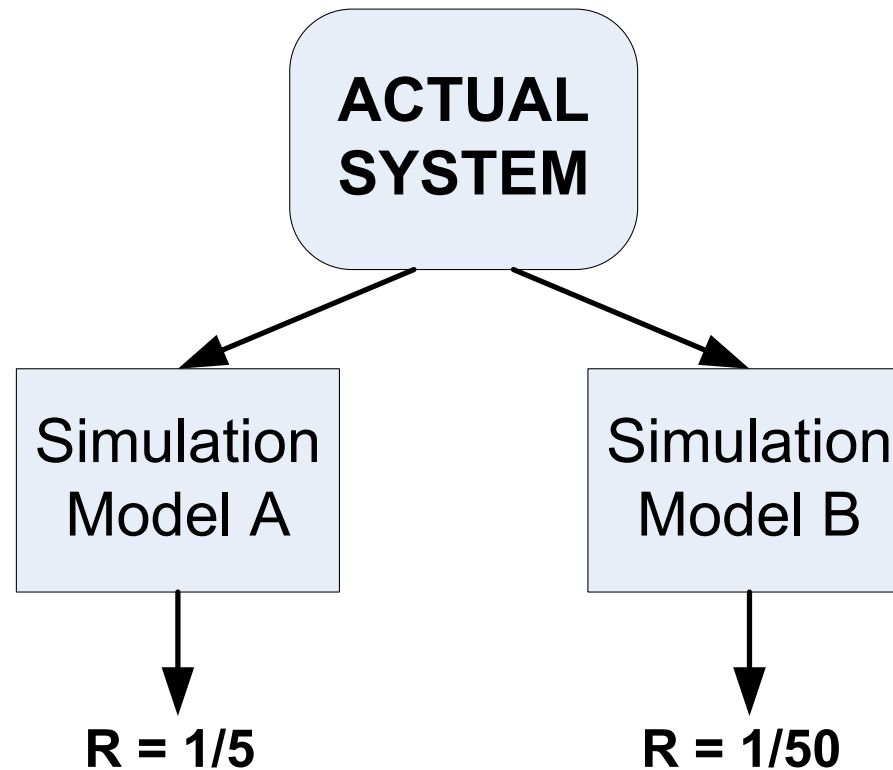
Example levels of detail

- Simulation model of a computer platform

Level of detail	Mean resolution time T	R ratio	Applications
Large-grained functional	50-500 ms	1/50 to 1/10	Analysis and preliminary design
Fine-grained functional	1-10 ms	1/10 to 20/1	Detailed design
Machine Instruction	1-10 μ s	20/1 a 100/1	Analysis and design of functional units
Digital logic	< 1 μ s	> 100/1	Logic design and diagnostics

Example levels of detail

- Same system, different simulation models



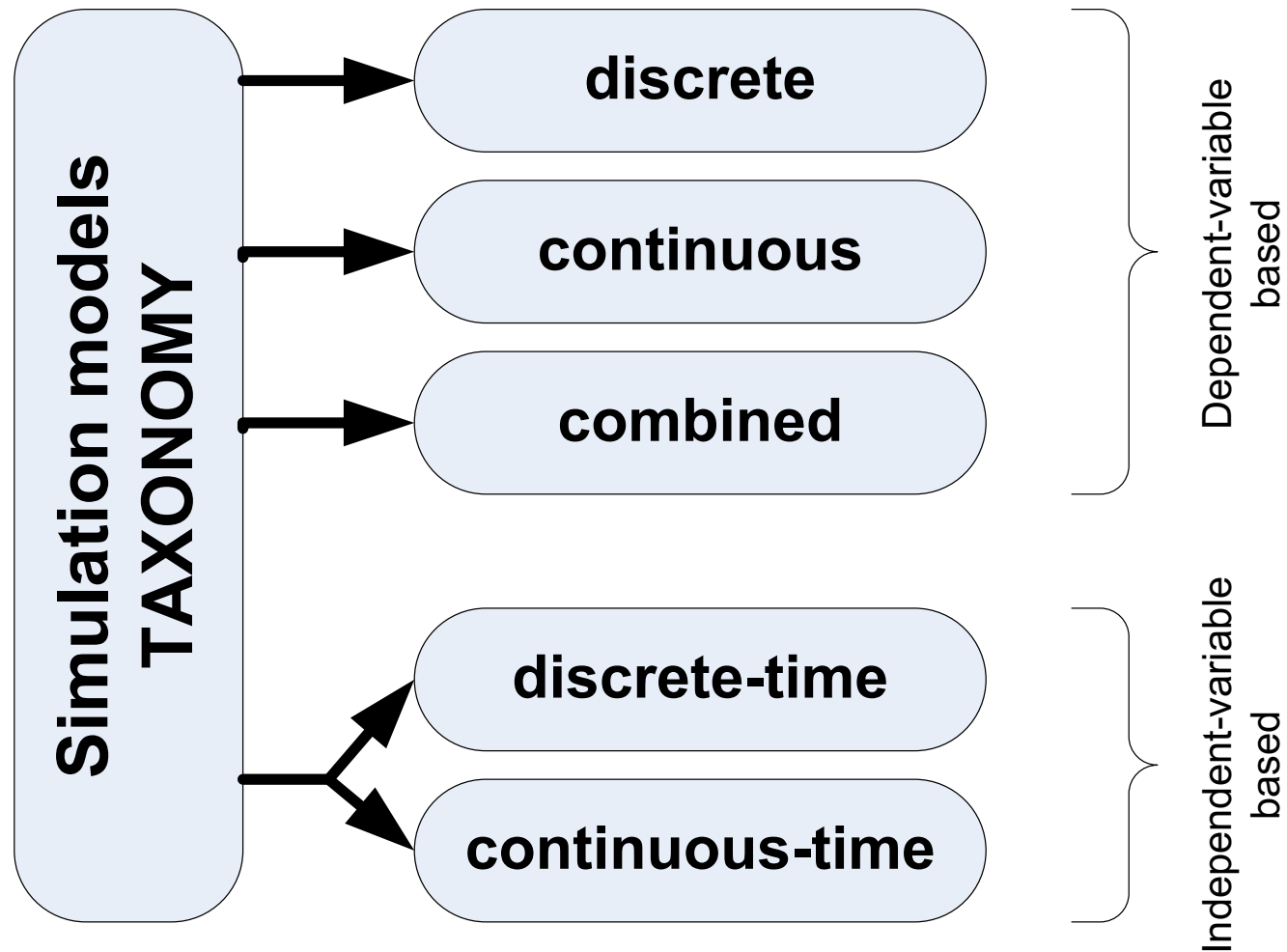
A's level > B's level

(provided both models are efficiently constructed)

Time flow mechanisms

- A simulation execution can be classified in terms of temporal aspects (e.g., a *real-time* simulation)
- As already mentioned, a second, independent, classification corresponds to the manner in which the state of the model changes as simulation time is advanced, sometime referred to as the *time flow mechanism*
- According to this second classification, simulation models may be broadly classified as **continuous** or **discrete**
- In a *discrete simulation*, the simulation model views the physical system as only changing state at discrete points in simulation time
 - conceptually the system is viewed as "*jumping*" from one state to the next, much like moving from one frame to another in a cartoon strip

Model taxonomy



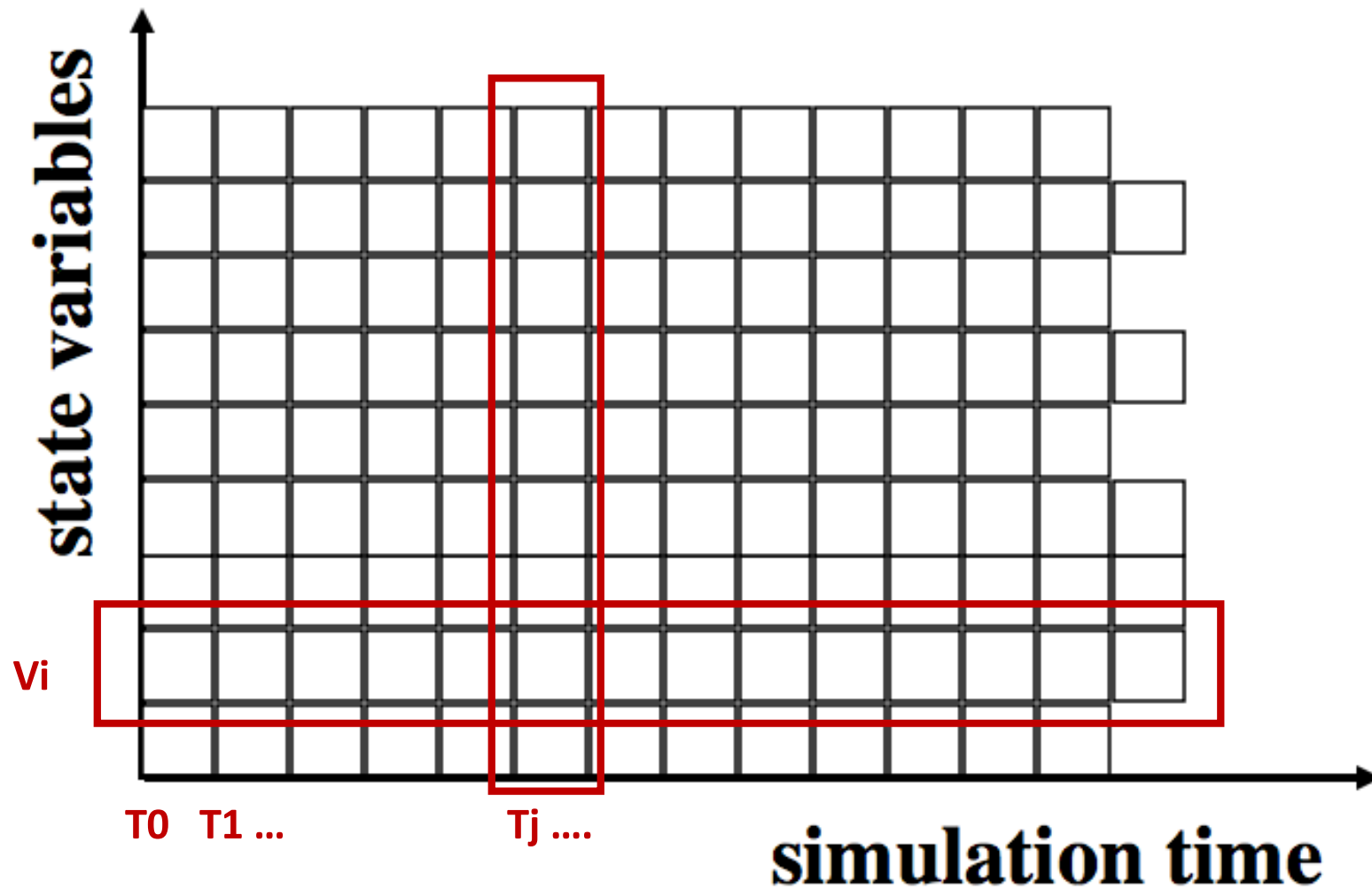
Time-stepped or Event-driven simulation

- The two most common types of discrete simulations are called *time-stepped* and *event-driven* (or sometimes called event-stepped) simulations
- These two categories of simulation are distinguished by the time flow mechanism used by the simulation to advance simulation time

Time-stepped simulation

- The simulation time is subdivided as a **sequence of equal-sized time steps**, and the simulation advances from one time step to the next
- Actions in the simulation occurring in the same time step are usually considered to be simultaneous
- If two actions have a causal relationship that must be accurately modeled in the simulation the actions must be simulated at different time steps
 - for example, an aircraft must vacate a runway before the next aircraft can land
- Thus, the size of the time step is important because it determines the precision of the simulation with respect to time

Space-time diagram for time-stepped simulation



Time-stepped *and* real-time simulation

- A variation on the time-stepped mechanism is sometimes used in real-time and scaled real-time simulations
- In a *real-time version time-stepped* simulation the program waits until wallclock time has advanced before advancing to the next time step

- Main loop

```
while (simulation in progress)
    wait until (W2S(wallclock time) > simulation
time)
    compute state of the system at the end of
this time step
    advance simulation time to the next time step
```